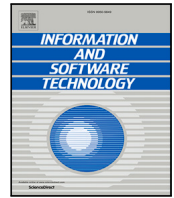




Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Service engineering for quantum computing: Ensuring high-quality quantum services

Ana Díaz ^a, Jaime Alvarado-Valiente ^b,* Javier Romero-Álvarez ^b, Enrique Moguel ^b,
Jose Garcia-Alonso ^b, Moisés Rodríguez ^a, Ignacio García-Rodríguez ^a, Juan M. Murillo ^b

^a Alarcos Research Group, Universidad de Castilla-La Mancha, Ciudad Real, Spain^b Quercus Software Engineering Group, Universidad de Extremadura, Cáceres, Spain

ARTICLE INFO

Dataset link: <https://github.com/javierrome236/quantumDeployment>

Keywords:

Quantum computing
Quantum software engineering
Quantum software quality
Process improvement

ABSTRACT

Context: Quantum computing is transforming the world and driving advanced applications in fields such as healthcare and economics. However, ensuring high-quality quantum software remains critical to its adoption across the industry. As quantum technology moves closer to practical applications, it faces significant challenges. Developers face platform-dependent complexities that make the creation of quantum applications a time-consuming process. In addition, the lack of mature tools further hampers progress and can compromise the quality of service.

Objective: The objective of this paper is to address the pressing need for quantum software quality assurance, presenting a solution for defining and using quantum services, by employing classical service engineering techniques and methods.

Methods: A process is presented for improving the generation, deployment, and quality assessment of quantum services using an extended OpenAPI Specification and the SonarQube tool. This process also integrates the automatic generation of code for the IBM Quantum provider and its deployment in containers ready for user consumption.

Results: After a detailed and individualized evaluation of the 40 implementations of quantum algorithms using the developed environment, the results reveal significant variability in the analyzability of the algorithms. This will serve in the future as a reference and guide for the continuous improvement of quantum algorithms in terms of their performance and efficiency in solving complex problems in various quantum application areas.

Conclusions: This research offers a fundamental contribution to the evolution of quantum computing by introducing a comprehensive framework for quantum software quality assurance. The proposed approach not only addresses some of the existing problems in quantum software, but also paves the way for the development of quantum algorithms and their servitization.

1. Introduction

The disruptive approaches that Max Planck or Niels Bohr established at the beginning of the 20th century on the principles of quantum mechanics, have produced innovations in many fields of science such as chemistry, optics, microelectronics, telecommunications or computation [1].

In the field of computing, quantum computing has evolved a great deal since pioneers such as Richard Feynman laid its first theoretical foundations in the 1980s [2]. Gradually moving from theory to practical application, more current progress have accelerated advances in the field, for example, through the more recent development of quantum hardware [3].

This advancement progress holds the promise of addressing complex problems, especially those that have traditionally been considered computationally insurmountable for classical computers within a practical timeframe, as it is the case with the challenges included in the Bounded-Error Quantum Polynomial Time (BQP) class [4]. Generalized interest in quantum computing is evident as technological giants like Amazon, IBM, and Google are investing heavily in developing quantum machines and making them accessible through their cloud platforms [5].

As quantum computing emerges as a paradigm shift, it shares commonalities with classical computing paradigms such as Service-Oriented Architecture (SOA). This convergence is the foundation for

* Corresponding author.

E-mail address: jaimeav@unex.es (J. Alvarado-Valiente).<https://doi.org/10.1016/j.infsof.2024.107643>

Received 31 October 2023; Received in revised form 18 October 2024; Accepted 2 December 2024

Available online 9 December 2024

0950-5849/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Quantum Computing as a Service (QCaaS) [6], where quantum computing resources are provided as services in conjunction with classical computing resources [7]. While this represents a significant leap forward, the development of quantum services has encountered several challenges.

In this way, the increasing demand for quantum services, particularly in hybrid architectures, highlights the pressing need for the development of tools and techniques that facilitate the achievement of desired quality levels in quantum software [8]. Therefore, quantum computing technology is rapidly advancing, with a growing interest from various sectors. However, it faces several challenges, including the low abstraction level at which developers are often required to work when creating quantum applications. This challenge arises due to the inherent complexity of quantum mechanics and the intricacies involved in quantum software development [9].

Consider a scenario in which a team of developers is assigned the task of implementing quantum algorithms for a commercial application. As they progress, they encounter various issues related to the maintainability and scalability of their quantum code, particularly as they need to switch between different quantum platforms. Without well-defined quality metrics or tools to assess the complexity and performance of their algorithms, the development process becomes slow, error-prone, and costly. Addressing quantum software quality early on would allow developers to avoid such pitfalls, ensuring that quantum applications are reliable, efficient, and adaptable as the technology evolves.

While it is true that the development of quantum software processes and lifecycles is still in its early stages, we argue that focusing on software quality at this point is not premature. Establishing quality metrics and evaluation frameworks now will ensure that as quantum software scales and moves into more complex, real-world applications, it will be robust, maintainable, and efficient. Furthermore, early adoption of quality standards can lead to better-defined development processes, creating a feedback loop that benefits both lifecycle management and quality assurance.

Several studies, such as those by [10,11], have focused on defining quantum software development lifecycles. Our study builds on these efforts by addressing an equally important aspect: software quality. By integrating tools like SonarQube and adapting classical quality metrics to quantum software, we aim to provide developers with the means to assess and improve their software from the earliest stages.

Furthermore, the integration and deployment of quantum software have proven to be problematic. The unique nature of quantum computing, which often relies on specific quantum platforms, poses challenges for seamless integration into existing classical systems. This results in developers struggling to create quantum applications that can run on various quantum machines. As a result, the development process becomes laborious and time-consuming [12]. Additionally, existing quantum development tools are still in their early stages, and their maturity is behind the rapid advancement of quantum hardware. These challenges collectively contribute to difficulties in maintaining the quality of quantum services, which is imperative for their successful real-world application [13].

To address these challenges effectively, our article introduces a novel approach. In this work, we propose the combination of an OpenAPI Specification¹ extension, its code generator, and the integration of SonarQube² within a Continuous Deployment approach driven by GitHub Actions³.

This proposal responds to the fundamental requirement for quantum services to meet high-quality standards [14]. The development of quantum services aims to create reliable, scalable, secure, and efficient

services that adhere to established quality benchmarks [3]. This is paramount in the context of quantum computing, as it can facilitate cost reduction, facilitate the identification and correction of issues at an early stage, enhance user experiences, and provide assurance of security and data protection [15,16].

In summary, our approach integrates the OpenAPI Specification and SonarQube with the objective of optimizing the quantum services generation and deployment processes, while ensuring the requisite quality standards are met.

The OpenAPI Specification offers a standardized, language-independent interface that clearly defines the quantum services' characteristics, including their functionality and parameters. This level of clarity ensures that quantum services are meticulously articulated and understood, minimizing potential ambiguities during their deployment. Moreover, SonarQube plays a pivotal role in reinforcing the quality of quantum services by providing automated source code analysis and identification of potential issues or vulnerabilities. This not only empowers developers to reach high standards of reliability but also bridges the gap between classical and quantum computing in hybrid architectures, where both technologies provide their resources through a SOA.

To guarantee the efficacy of our proposal, we conducted an evaluation utilizing 28 quantum algorithms, with a total of 40 implementations. The objective of this evaluation was to confirm the correct generation and deployment of the algorithms in question, while also assessing the quality of the code using a set of quantum quality metrics. This comprehensive approach represents a significant step forward in the development and deployment of quantum services, thereby contributing to the expansion of the quantum software ecosystem.

The details of this process are outlined in the remainder of the document as follows: Section 2 presents an analysis of the background of the presented work, a discussion of the most relevant related works, and an address of the importance of the so-called quantum quality. In Section 3, we present our process for the automated generation and deployment of quantum services, along with a method for quality assurance and code analysis using quality metrics. In Section 4, we discuss the outcomes of our method's comprehensive evaluation. Finally, Section 5 summarizes the most important results and draws conclusions from the work.

2. Background

In the field of quantum software development, ensuring the quality and reliability of quantum services is of crucial importance. However, despite the ongoing efforts to define software life cycles and quality models for quantum applications, a standardized approach for quality evaluation remains elusive. This section delves into the emerging field of Quantum Service-Oriented Computing (QSOC) and the imperative need for tools to enhance the quality of quantum services, setting the stage for the subsequent exploration of quantum quality measurement and its integration into the development process.

2.1. Quantum service-oriented computing

The current dearth of Software Engineering techniques suitable for quantum services is affected by multiple levels, such as the low abstraction capacity that developers must face and the absence of mechanisms for integration, deployment, or quality assurance of the produced software [17]. To this end, within the evolving landscape of quantum software development, significant efforts have been made to introduce systematic methodologies to improve the quality and reusability of quantum applications.

A pertinent example is the work of B. Weder et al. [18], who propose a comprehensive quantum software development lifecycle comprising multiple phases. This lifecycle systematically applies Software Engineering principles, spanning from the initial requirements analysis to

¹ <https://swagger.io/specification>

² <https://docs.sonarqube.org/latest>

³ <https://docs.github.com/en/actions>

implementation, with the overarching goal of ensuring the quality of quantum software. Although this approach adheres to established Software Engineering best practices, it currently lacks conformity to established norms or quality standards, which may introduce challenges in its practical implementation.

Thus, to address these challenges and establish a standardized approach to assessing quantum quality, we turn to the emerging field of QSOC. This approach provides a foundation for efficient quantum software development. In this domain, notable works such as Kumara et al. [19] propose a reference architecture for the Quantum Service-Oriented Architecture (QSOA) model that adopts the layered SOA model. Moreover, they present a model-driven methodology for the realization of QSOA applications, which facilitates the construction of portable and adaptable QSOA applications and includes an enhancement for the migration of classical algorithms and services.

Alongside these works, the success of QSOC has been considerably enhanced by the simultaneous development of cloud computing as a revolutionary paradigm. This innovation aims to provide dynamic, reliable, and adaptive computing environments [20]. Thus, several fundamental reasons have contributed to the success of the cloud. Cloud computing allows quantum companies and developers to exercise greater cost control. Traditional quantum computing requires significant investments in hardware and infrastructure. With quantum cloud platforms, companies can mitigate these expenses, as they do not need to purchase, upgrade, and maintain quantum hardware. Instead, they pay for the quantum computing resources they use. In addition, quantum cloud service providers offer unmatched flexibility and scalability, which are essential in the quantum domain [21].

In light of these considerations, a solution for running quantum services in the cloud, referred to as Quantum API Gateway, is put forth by Garcia-Alonso et al. [22]. In this proposal, the authors discuss the challenges of combining quantum algorithms with traditional software and suggest that the Software Engineering methods and abstractions needed for complex solutions that combine both types of software are not yet consolidated.

However, while the foundation of these works and solutions align with established Software Engineering best practices and the use of the tools of the cloud providers, it is important to highlight that they currently lack conformity to established norms or quality standards. This discrepancy underscores a pressing challenge within quantum software development. The absence of standardized quality evaluation approaches introduces a layer of complexity in the practical implementation of these meticulously crafted methodologies [16]. Hence the need for Quantum Quality developments.

2.2. Quantum quality

Software quality is a critical aspect of software development. It refers to the degree to which the software meets the specified and implicit needs of users, as well as its ability to function reliably and efficiently in a given context [23].

One of the fundamental objectives in the technology industry is the development of quality software. In order to achieve this, it is necessary to have quality standards, such as the ISO/IEC 25010 standard [24], which provides the set of criteria and specifications established to measure and evaluate the quality of a software product. This ensures consistency, security, reliability, and competitiveness, facilitating cost reduction, compliance with regulations, and promoting continuous improvement. The ISO/IEC 25010 standard defines eight fundamental characteristics to determine the overall quality of the software product, one of them being maintainability; that is, the ability of the software to be modified and improved efficiently [25]. Maintainability in turn encompasses several sub-characteristics, one of which is analyzability. Analyzability refers to how easy it is for developers and testers to analyze and understand the software. When software exhibits a high degree of analyzability, it is straightforward to examine, diagnose

potential issues, and implement improvements or enhancements. This fact results in more efficient maintenance, which in turn reduces costs and the time spent on the process.

Below are the properties used to evaluate the analyzability of quantum software, which have been normalized in values from 0 to 100, with 0 being the lowest quality value and 100 the highest value. These quality properties are based on [26]. The properties focus on the quantum software representation and the ability to perform static analysis in the evaluation environment.

- **Circuit Width (CW).** Indicates the number of qubits required to represent and process information in a quantum circuit. A circuit with a larger number of qubits may require more resources and be more complex to analyze.
- **Circuit Depth (CD).** Refers to the number of layers of operations that make up a quantum circuit. A greater depth can increase the complexity of the circuit and thus make it more difficult to read.
- **Complexity of circuit gates (CCG).** Evaluate the number and type of quantum gates used in a circuit⁴. More complex gates, applied over a larger number of qubits, can increase the difficulty of understanding and modifying the circuit.
- **Conditional instructions (CI).** This refers to operations that are executed in a quantum circuit based on the result of previous measurements. This adds complexity and may require slower and more detailed analysis for circuit understanding and analysis.
- **Quantum cyclomatic complexity (QCC).** This property evaluates the complexity of a quantum program in terms of the number of possible logic paths through the quantum code [27]. Higher cyclomatic complexity may indicate a circuit that is more difficult to analyze.
- **Measurement operations (MO).** These are instructions that measure the quantum state of one or more qubits in a quantum circuit. Although they may be fundamental at some specific point, they increase the depth of the circuit and thus the complexity of analysis.
- **Initialize and reset operations (IRO).** They refer to the operations that prepare the initial state of the qubits before the execution of the circuit and those that reset them to a known state after the execution. Trying to understand how qubits are set and reset can complicate the understandability of the circuit.
- **Auxiliary qubits (AQ).** These are additional qubits that are temporarily used to perform certain operations of a circuit. These require additional analysis to understand how they are used and affect the behavior of the circuit.

The value of the analyzability will depend on the value obtained for each of these properties, also taking a normalized value from 0 to 100. To assign the value to the analyzability, a function by profiles is used [28]. For this purpose, a profile-assigning function is used, in which levels are used to classify the properties that influence the evaluation according to their level of quality. Ranges set the maximum number of properties that are allowed at each level. Each rank has a range of quality scores associated with it, and the exact value is determined by considering the difference between the values of the system profile and the maximum values defined in the rank in which the system is located, as well as the values defined in the next higher rank.

As Quantum Software Engineering advances, it seeks to address the challenges of quantum software development by providing comprehensive concepts, principles, and guidelines [29]. However, there is also a need to establish standards and best practices in quantum software development as confirmed by over four hundred researchers, developers, and companies in The Talavera Manifesto for Quantum

⁴ https://qiskit.org/documentation/apidoc/circuit_library.html#standard-gates

Software Engineering and Programming [30], seeking to ensure that quantum computing is driven by robust and reliable algorithms.

Although good practices are being established for quantum software development and maintenance, such as version control and testing frameworks [31], a critical challenge arises related to the lack of maturity in tools and insufficient infrastructure to adapt, customize, automate, and configure agile practices in quantum software development [32]. In the agile context of software development, especially in DevOps, where adaptability and fast delivery are crucial, static code analysis becomes an effective method to identify real-time improvement opportunities [33].

3. Provision of quality quantum services

In this section, we explore the quantum services development process, covering the stages from circuit conception and design, through validation and evaluation, to service deployment. Thus, this process involves a series of interrelated steps that collectively define the development and operational aspects of quantum services.

3.1. Methodology

This study follows a methodology based on the use of service engineering techniques adapted to the context of quantum software development. First, we have defined a set of key research questions that guide the work:

1. How can classical service engineering techniques be adapted for quantum software development?
2. What are the key quality metrics in quantum software, and how do they vary across different quantum algorithms?
3. How effective is the combination of OpenAPI, SonarQube, and GitHub Actions in automating the development and quality assurance of quantum services?

To address these questions, we implemented an approach based on extending the OpenAPI Specification, along with code quality analysis tools like SonarQube and continuous deployment automation through GitHub Actions. Throughout the process, we evaluated 40 implementations of quantum algorithms using specific metrics to measure quantum software quality, such as quantum cyclomatic complexity and circuit depth. The methodology focuses on both the automatic generation of quantum services and the continuous assessment of their quality, thus fostering an agile and high-quality development environment for quantum algorithms.

3.2. Generation and deployment of quantum services

The development of fully operational quantum services necessitates not only the incorporation of business logic but also a seamless comprehension of the service's characteristics. While documentation and source code are undoubtedly valuable resources, they may not always offer a comprehensive overview. To address this challenge, we employ the OpenAPI Specification, which allows us to define the service and its parameters using a standardized, language-independent interface designed for RESTful APIs [34].

Additionally, this proposal is focused on Continuous Deployment, using GitHub Actions, a GitHub feature that allows developers to automate various tasks and workflows in their software development process. So, GitHub Actions facilitates the automation of time-consuming tasks, streamlining the release of software updates and the deployment of the quantum services in containers, thus enhancing accessibility for users.

3.2.1. Generation of quantum services using the OpenAPI Specification

The OpenAPI Specification is instrumental, much like in classical service creation, for defining and generating quantum services.

In addition to OpenAPI, RESTful API Modeling Language (RAML) and API Blueprint are among the most widely used standardized languages for the definition of API Rest. However, due to the limitations

they present when generating code [35], OpenAPI has been used for the specification and generation of quantum services. For this purpose, we have adapted the OpenAPI Code Generator⁵, a tool that allows the generation of API libraries and server stubs, tailoring it to support quantum applications. [The tool incorporates a variety of extensions, each tailored to facilitate code generation for distinct programming languages. Consequently, a new extension has been developed to enable the generation of quantum services in the Python language.] This extension introduces custom properties and augments the code generator to accommodate quantum-specific functionalities. It standardizes the process of defining quantum services, enabling the automatic generation of quantum services from an API specification and a quantum circuit [36].

Fig. 1 illustrates the complete process, which has been developed with consideration for the notation Business Process Execution Language (BPEL). This notation is essential in the representation and description of business processes in this context [37]. Thus, we have incorporated BPEL notation to structure and define the key processes involved in the generation, deployment, and validation of quantum services. The complete process includes a manual process (Task 1), i.e. tasks that require the developer's hand, as well as automatic process (Task 2).

Thus, at the start of the manual process (Task 1), the first step is to integrate the business logic into the quantum services, and for this we use Quirk⁶, a web-based quantum programming tool. Quirk offers a user-friendly graphical interface for designing quantum circuits by employing drag-and-drop operations. The software was created in JavaScript and allows users to create circuits more rapidly while providing programmatic access to the quantum circuits that are generated.

Moreover, Quirk is designed to be flexible, allowing for the integration of any quantum circuit creation tool that offers programmatic access to code. This flexibility ensures compatibility with the OpenAPI Specification.

As an alternative, we provide the option to directly incorporate quantum circuits into the service endpoint through a URL pointing to the repository where the circuit is defined, like this example⁷, thus accommodating more experienced developers who prefer code-based circuit creation. The URL, containing the quantum circuit in IBM Qiskit language⁸, acts as a crucial input for the subsequent phases of the pipeline.

The OpenAPI Specification plays a pivotal role in defining the service's endpoint. It provides a standardized, language-independent interface for RESTful APIs, simplifying communication between users and services. To address quantum services development, we have extended the OpenAPI Specification. The extended specification includes custom properties that cater to quantum applications, allowing developers to include quantum-specific details. Some of these custom properties include the specification of the URL of the quantum circuit, the quantum provider for which the code is being generated, or attributes that can be used at runtime, such as the number of shots and the quantum machine where the code will be executed.

Once this manual phase of the process is completed, we obtain as an artifact of this phase the YAML file, which includes the extended specification and the service endpoints, incorporating information related to the code to be encapsulated or the URL that directs to the circuit designed within Quirk.

Upon completion of the manual process, an automatic process is initiated (Task 2), which subsequently bifurcates into two parallel processes: continuous deployment (Task 3) and quantum quality through code analyzability (Task 4). The following sections will provide detailed explanations of these processes.

⁵ <https://openapi-generator.tech>

⁶ <https://algassert.com/quirk>

⁷ https://bitbucket.org/spilab/quantum-circuits-code/src/main/Popular/Shor_algorithm.py

⁸ <https://qiskit.org>

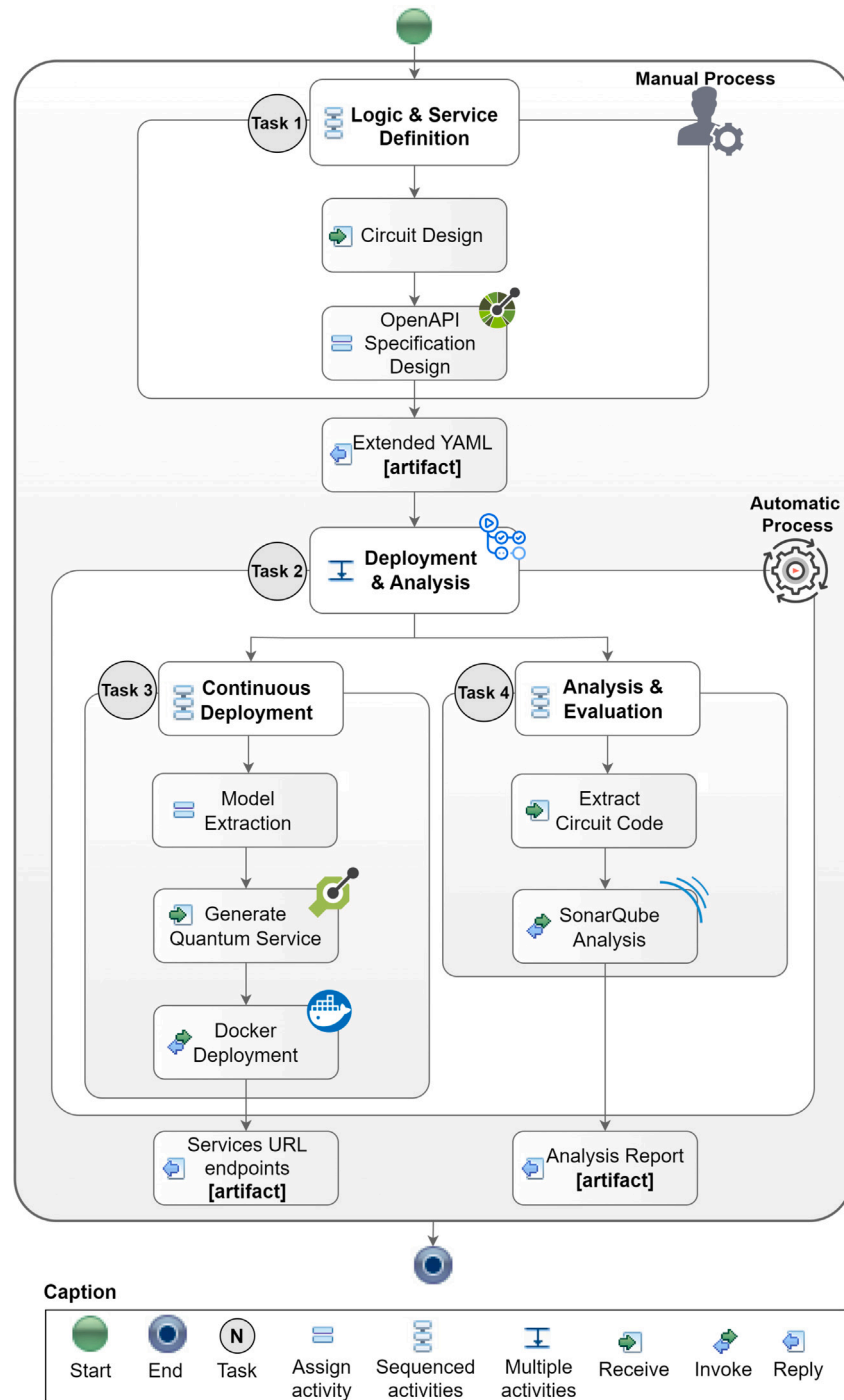


Fig. 1. Quantum quality service delivery process: generating, deploying, and ensuring quantum services quality, including a manual process (Task 1) for service definition and circuit design, followed by an automatic process (Task 2) that includes automated deployment (Task 3) and quality analysis (Task 4).

3.2.2. Using GitHub actions and docker to streamline quantum deployment

In the next phase of our process, we focus on automating the deployment of quantum services (Task 3). This approach ensures systematic deployment through Continuous Deployment with the help of GitHub Actions and Docker software containers. This streamlining of the deployment process not only enhances accessibility but also significantly reduces manual intervention.

The process initiates with the execution of GitHub Actions, as defined in the repository⁹ where the process is configured. It is crucial to

understand that this automated process commences when the developer performs a *commit* to the YAML with the specification of the services, effectively marking the completion of their design work. Thus, the Continuous Deployment approach, enabled by GitHub Actions, stands out for its ability to free developers from time-consuming tasks, accelerate software releases, and automate the deployment of containerized services. Ultimately, the automation facilitates greater convenience for users in accessing and utilizing the services.

In this automatic process, the OpenAPI Code Generator, equipped with the extended specification explained before, assists in generating source code for quantum services. This source code implements

⁹ <https://github.com/javierrome236/quantumDeployment>

quantum API endpoints in Python, using Flask¹⁰ as a web application framework. These endpoints are responsible for deploying and executing quantum algorithms on specified providers and machines, based on the custom properties integrated into the OpenAPI Specification.

A source code generator is employed to automatically generate the code structure in accordance with this specification. Developers can choose their preferred programming language and integrate business logic, creating a fully functional service. This approach fosters an efficient and seamless exchange of information between different software systems.

Once the services are automatically generated and deployed in a Docker container, the URL with the endpoints of those services is obtained as an artifact. The availability of the deployed services is exposed through one of the server ports. To this end, the system performs checks on a predefined list of available ports and assigns the first one to open. It is essential to note that, in the current context, services can only be deployed on the same server on which the entire system is hosted. After this process, the user immediately receives a link to the server, together with the designated port number, which facilitates access to the generated services.

This multi-faceted approach, incorporating the OpenAPI Specification, code generation, and automated deployment with GitHub Actions, lays the foundation for efficient and standardized quantum services development, promoting accessibility, reliability, and quality in quantum software.

The Continuous Deployment process is also complemented by a code validation and analysis phase, using a tool called SonarQube, as explained in the next section. This approach ensures that our quantum services not only remain accessible and efficient but also meet the highest quality and security standards.

3.3. Quantum quality through code analyzability

It is important to clarify that this work focuses on software quality metrics, particularly regarding the analyzability and maintainability of quantum algorithms. These metrics aim to ensure that quantum algorithms are easy to understand, maintain, and improve. The scope of this study is centered on ensuring maintainable and high-quality software.

To ensure a comprehensive evaluation of code quality and the identification of potential issues and defects, we have integrated SonarQube into our quality service delivery process (Task 4), as illustrated in Fig. 1.

SonarQube is a highly regarded tool in both industry and research circles, renowned for its capabilities in conducting static code analysis and assessing software quality. The incorporation of the SonarQube parser confers a distinctive advantage, whereby quantum metrics analysis is integrated into a well-established development and analysis environment. The use of SonarQube presents numerous key benefits, including its seamless integration into our existing workflow, a dedicated plugin for quantum metrics evaluation, ease of use, and adaptable configuration to cater to project-specific requirements. Furthermore, the plugin guarantees the straightforward incorporation of quantum software quality measurement from a multitude of initiatives, thus rendering our approach more inclusive and accessible.

SonarQube is an industry-standard tool used for static code analysis. Its primary function is to identify potential issues such as bugs, vulnerabilities, and code smells. In the context of quantum software, SonarQube was adapted using a custom plugin designed to extract quantum-specific metrics. This tool facilitates continuous quality assurance through the analysis of quantum circuits at each stage of the development process.

SonarQube performs static analysis on the quantum algorithms implemented in Qiskit and Quirk. The integration of a custom quantum

metrics plugin enables the tool to automatically extract key quality metrics from the quantum circuit code. These metrics include circuit width (CW), circuit depth (CD), quantum cyclomatic complexity (QCC), and gate complexity. Upon each commit, SonarQube parses the code, calculates these metrics, and compares them against predefined thresholds to evaluate the code's analyzability and maintainability. This automated analysis ensures that the quantum software meets high-quality standards throughout the development process.

After the analysis is completed, SonarQube generates detailed reports highlighting the strengths and weaknesses of the quantum circuits. These reports include visualizations of the quantum metrics, which assist developers in identifying potential areas for improvement. By integrating SonarQube into the GitHub Actions pipeline, the analysis is triggered automatically with each code change, ensuring real-time feedback and continuous quality monitoring.

Our primary objective is to provide detailed quality metrics that indicate potential code problems and enable their resolution. To achieve this, our process, conducted in parallel with the Continuous Deployment steps, extracts files exclusively containing circuit code. These files are securely stored in a separate repository, and another set of GitHub Actions automatically subjects them to SonarQube's analysis.

The integration of GitHub Actions with SonarQube enables the automation of circuit code analysis with each *commit* or *pull request*, delivering comprehensive reports on code quality and potential security concerns. This meticulous assessment ensures that the code destined for production meets the rigorous quality and security standards set by the industry.

The initial step is configuring SonarQube on a dedicated server. It is imperative to note that within the realm of software quality analysis, SonarQube stands as one of the most widely embraced tools. However, it is essential to acknowledge that, given the incipient state of quantum software metrics, SonarQube currently lacks a dedicated measurement system for quantum software. To bridge this gap, in the context of measuring quantum algorithms, a plugin has been developed within AQCLab¹¹ (the first laboratory accredited for software quality assessment under ISO/IEC 25000) that is particularly valuable in providing specific metrics for evaluating the code of quantum programs.

Integrating the plugin into SonarQube for the continuous evaluation flow is a straightforward process. Primarily, the plugin must be accessible via its source code. Subsequently, the plugin is uploaded to the SonarQube server via the administration interface. Once uploaded, the plugin becomes an integral component of the platform, with its functionalities becoming available for configuration and utilization in development projects.

This flexibility in integration allows SonarQube to be tailored to specific code analysis needs, including the measurement of quantum algorithms developed using Qiskit. Qiskit is an open-source toolkit developed by IBM for programming and executing quantum algorithms and was selected for the development of the quality assessment environment for quantum software because it is widely used by the quantum computing community and is currently the most advanced quantum development kit.

This paper presents the mentioned plugin and its metrics, as well as two tools (also developed within AQCLab) that serve to: (i) give an evaluation of the analyzability based on the results of the metrics; (ii) have a user-friendly visualization of the results, to complete the process of evaluating the analyzability of quantum software. To facilitate the understanding of the architecture of the analyzability assessment tools, Fig. 2 shows the existing relationships between them.

The first tool consists of a Python program that automatically evaluates the product based on the quality model built, based on ISO/IEC 25000, and stores the evaluation results in a database. The second tool, based on PowerBI, subsequently retrieves the evaluation results from

¹⁰ <https://flask.palletsprojects.com>

¹¹ <https://aqclab.es>

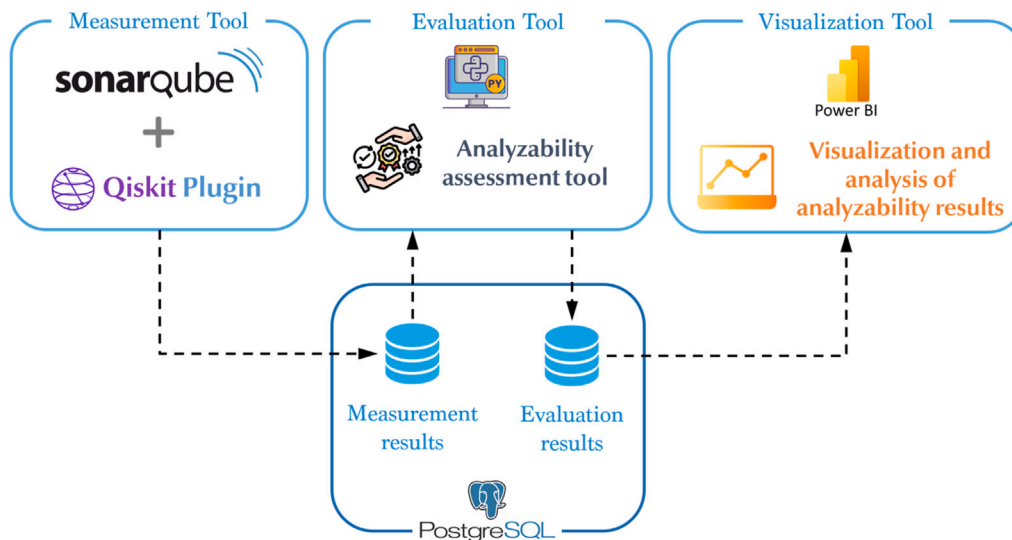


Fig. 2. Architecture of quantum code analyzability tools and their integration for metric evaluation and visualization.

the database and displays them to the evaluator, thus facilitating their comprehension using panels, tables, and comparative graphs on the results of both the measurement and the evaluation.

Other tools, such as QChecker, have been developed for the static analysis of quantum programs. These tools are designed to detect specific bugs and to ensure compliance with quantum programming practices. Nevertheless, SonarQube presents notable advantages, including the capacity to develop custom plugins for quantum-specific metrics, comprehensive community support, and seamless integration into continuous development workflows. The combination of these two tools can facilitate an enhanced quality assurance process for quantum software, leveraging the adaptability and comprehensive analysis of SonarQube in conjunction with the specialized capabilities of QChecker [38].

3.4. Findings and key contributions

This study presents an innovative quality-based paradigm for quantum services generation and deployment. This transformative approach results in several significant contributions: Our analysis dissects the intricate challenges faced by developers working on quantum applications. We unearth the multifaceted complexities of quantum software development.

We propose a process for automatic generation and deployment of quantum services. This process is a direct response to the identified challenges, offering an automated, systematic, and streamlined approach to quantum software development. It significantly reduces manual intervention, minimizes error rates, and optimizes the quantum services deployment process.

The objective of our study is to automate the assessment of the quality of quantum software through the integration of tools like SonarQube, which allows for continuous monitoring of metrics such as quantum cyclomatic complexity. This approach complements previous work on quantum software development lifecycles, particularly the lifecycle models proposed by [10]. Their lifecycle emphasizes the hybrid nature of quantum and classical software development but lacks a detailed focus on real-time software quality evaluation during the development process [10]. Our findings bridge this gap by providing a practical framework that assesses software quality as the application evolves, ensuring maintainability and analyzability from the early stages of development.

Additionally, the QSOC methodology introduced by [11] presents a model-driven approach for developing hybrid quantum-classical applications, with an emphasis on service composition and infrastructure

management [11]. While their work is primarily focused on the orchestration and resource optimization of quantum services, our study builds on this by offering a concrete implementation of quality assurance mechanisms for quantum algorithms through the use of static code analysis tools. This not only facilitates the integration of classical and quantum services but also enhances the reliability and scalability of the developed applications.

The quality analysis we have introduced raises the standards of quantum services development. It encompasses the formulation of tailored quality metrics for quantum software and the use of a state-of-the-art monitoring tool. This integration improves quality control and real-time monitoring of quantum services, promoting excellence in quantum software.

By comparing our findings with these established works, we demonstrate that integrating automated quality evaluation into the quantum software lifecycle is essential for ensuring high-quality quantum applications in real-world settings.

Therefore, our empirical evaluation confirms the effectiveness of our approach. It determines the code's levels of analyzability and ensures the delivery of quantum services, as described in the following section.

Finally, the implementation of quality assurance practices in quantum software not only improves the reliability and maintainability of algorithms but also can have a significant impact on the future development of quantum computing. As the industry progresses, establishing clear quality standards will facilitate collaboration between researchers and developers, creating a more robust ecosystem for innovation. For example, in sectors like cryptography and artificial intelligence, where precision and efficiency are critical, adopting specific quality metrics for quantum software can accelerate the adoption of these emerging technologies, ensuring that quantum applications are more reliable and accessible for implementation in real-world environments. This will not only contribute to the evolution of the field but also enable companies to leverage the full potential of quantum computing capabilities.

4. Use case of the evaluation environment

To validate the technology and its benefits, in this section, we address the crucial evaluation process of determining the level of analyzability using a set of 28 quantum algorithms.

In the context of quantum computing, the term 'analyzability' is used to describe the ease with which an algorithm can be understood and analyzed in terms of its inner workings. This, in turn, influences the maintainability of the algorithm. This evaluation phase represents

a fundamental milestone in the validation and comprehension of quantum algorithms, providing valuable information on their feasibility and usefulness in real applications.

This section provides a comprehensive account of the methodology employed for the evaluation of analyzability of the algorithms and the ensuing results. This presentation offers a more comprehensive understanding of the performance of this type of environment in the field of quantum computing.

4.1. Quantum algorithms

The selection of the 28 quantum algorithms was based on their relevance to the field of quantum computing, covering a wide variety of problems and applications. Widely recognized algorithms, such as the “Shor” and “Grover” algorithms, have been included, as well as other algorithms that are less well known but relevant in specific contexts, such as optimization and quantum cryptography. Thus, this selection covers from the factorization of large numbers to the resolution of Pell equations and search optimization, representing significant advances in the ability to solve complex problems by using quantum computation.

Furthermore, 40 implementations of these algorithms have been analyzed. The utilization of disparate implementations of these algorithms enables the assessment of quality of different variations and adaptations of these algorithms, thus covering a more representative range. Moreover, the existence of multiple implementations and versions in different simulation environments suggests a comprehensive approach to development and testing to maximize the efficiency and accuracy of these algorithms.

In addition to their relevance, the quantum algorithms used in this study were sourced from widely recognized community resources, particularly the Qiskit library developed and maintained by IBM. Qiskit offers a comprehensive set of pre-built quantum algorithms, which are openly available and commonly used by the quantum computing community. Additional algorithm implementations were obtained from open-source repositories hosted on GitHub, allowing us to leverage well-established quantum software frameworks and contribute to the reproducibility and reliability of our results.

To ensure consistent and accurate data collection, we utilized the SonarQube tool, integrated with a quantum-specific plugin developed for this study. This setup enabled the automated extraction of key quantum software metrics, including quantum circuit width, depth, and cyclomatic complexity.

The quantum algorithms that have been used as a use case for applying the evaluation tools are as follows:

1. Hidden algorithm
2. Abelian algorithm
3. Shor algorithm
4. Primality algorithm
5. Polynomial algorithm
6. Phase Estimation algorithm
7. Searching algorithm
8. Gauss algorithm
9. Less11 algorithm
10. Less6 algorithm
11. Range2 algorithm
12. Range1 algorithm
13. Deutsch-Jozsa algorithm
14. Full Adder algorithm
15. Pell algorithm
16. Shor algorithm
17. Bernstein-Vazirani algorithm
18. Simon algorithm
19. Kickback algorithm
20. Quantum Fourier Transform (QFT) algorithm

21. Grover algorithm
22. Teleportation algorithm
23. Subset algorithm
24. Subset Search algorithm
25. Ordered algorithm
26. Decoding algorithm
27. Traveling Salesman Problem (TSP) algorithm
28. Quantum Approximate Optimization algorithm (QAOA)

The assessment of the quality of these algorithms is crucial to determine their usefulness and applicability in real-life scenarios. Each of them may have specific advantages and disadvantages in terms of analyzability and maintainability, which makes detailed evaluation essential for selecting the optimal tool for a given problem.

4.2. Results of the analyzability evaluation

This section presents the results of the evaluation of the analyzability of the quantum algorithms presented in the previous section, based on measurements performed with the help of SonarQube and a specific plugin for Qiskit. The latter, called “QiskitPlugin”, plays a crucial role in providing the essential metrics needed for the evaluation of the analyzability properties of quantum algorithms.

4.2.1. Measurement results

The initial phase of the evaluation entails the undertaking of comprehensive measurements of the quantum algorithms that have been implemented within both the Qiskit and Quirk development environments. In order to achieve this, the SonarQube static code analysis tool is employed, along with the central component of the measurement, the “QiskitPlugin”.

The metrics encompass a range of characteristics, including the number of circuits and the number of qubits in each quantum circuit, as well as the cyclomatic complexity of the algorithms and the complexity of the gates utilized, contingent on the number of qubits necessary for their computations. This enables an assessment of the simplicity with which the algorithms can be comprehended and analyzed.

The results obtained for the width and depth properties during this first phase of measurement are shown in Fig. 3 for the circuits developed in Qiskit.

In terms of analyzability, the results show significant variability among the quantum algorithms evaluated. It is highlighted that algorithms such as “Hidden” and “Abelian” show a higher complexity in terms of width and depth, suggesting that they may require additional effort in their analysis and comprehension. On the other hand, algorithms such as “Kickback” present a simpler structure, with equal width and depth and low values, making it relatively simple to comprehend.

The results obtained after measurement for the width and depth properties are shown in Fig. 4 for the circuits developed in Quirk.

The measurement results on the circuits developed at Quirk show an interesting trend. Generally, it is observed that most of the algorithms have a relatively simple structure in terms of width and depth. This indicates that these circuits may be easier to comprehend and analyze compared to more complex Qiskit circuits.

On the other hand, the algorithm of “Grover” shows greater depth compared to others, which may indicate greater complexity in its inner workings. While “Simon” exhibits the greatest breadth. This may require a more detailed analysis to fully understand its behavior.

The “QAOA” algorithm stands out for its low depth, value 1, and width, value 2, suggesting a relatively simple implementation as far as these two properties are concerned.

Another property that is interesting to analyze after measurement is the quantum cyclomatic complexity for the Qiskit algorithms (see Fig. 5), an important metric that helps us evaluate the structure and complexity of quantum algorithms.

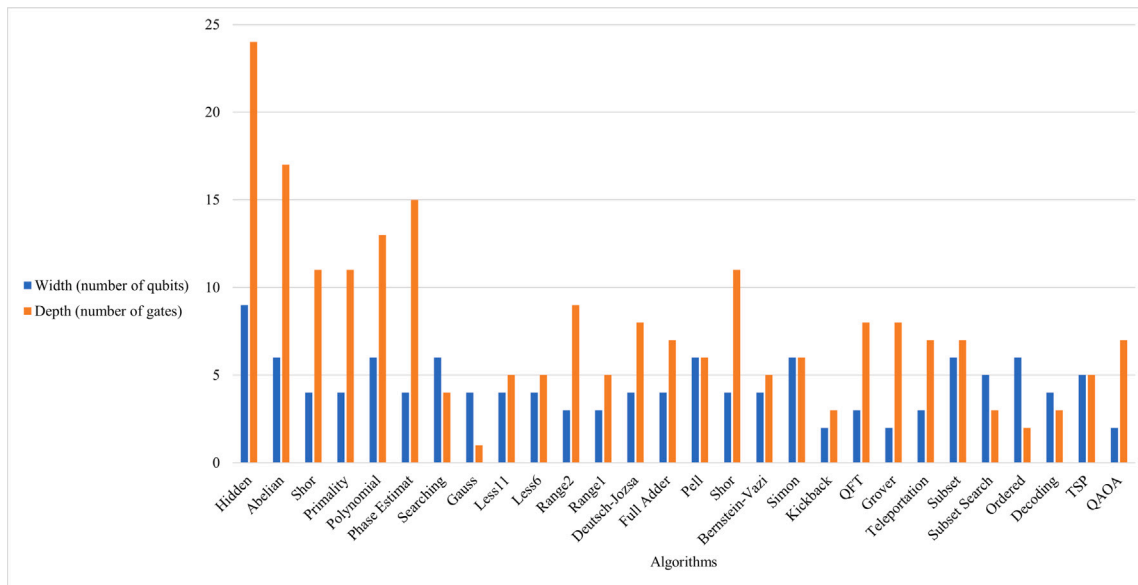


Fig. 3. Width and depth measurement results of quantum circuits developed in Qiskit.

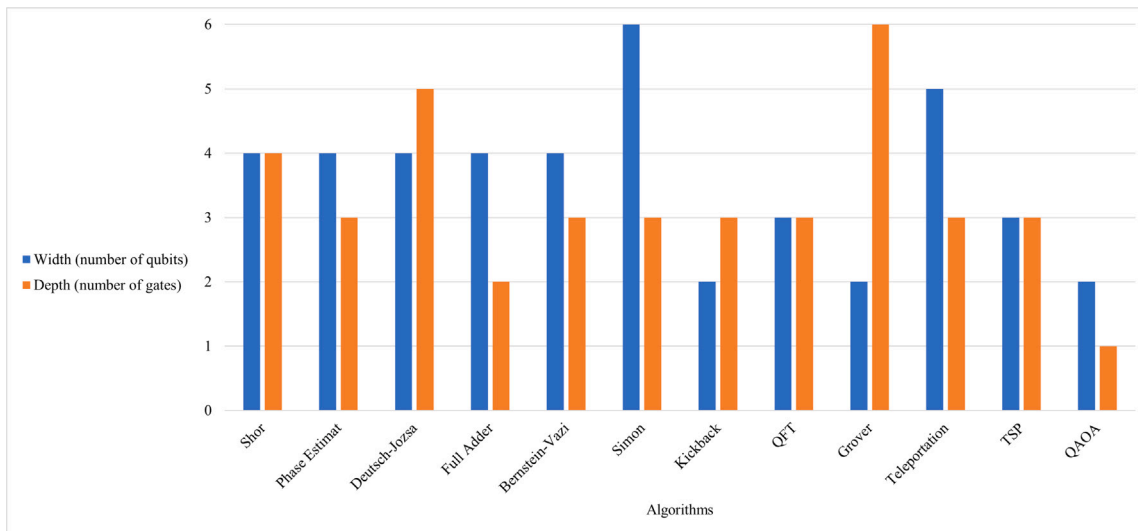


Fig. 4. Width and depth measurement results of quantum circuits developed in Quirk.

Overall, we observe a wide variability in the cyclomatic complexity of the algorithms. Some algorithms exhibit a more complex algorithmic structure, suggesting that they may require more detailed analysis to fully understand their operation. This is evident in algorithms such as “Hidden”, which exhibits a cyclomatic complexity of 79, indicating a significantly complex algorithmic structure.

On the other hand, some algorithms exhibit lower cyclomatic complexity, even reaching zero, suggesting a simpler and more linear structure in comparison. These algorithms may be more accessible for analysis and comprehension, which may be beneficial in terms of implementation and optimization.

However, the results provided by the “QiskitPlugin” for the quantum cyclomatic complexity of the algorithms developed in Quirk are equal to 0. This indicates that these algorithms have a very linear and simple structure from a cyclomatic complexity point of view, suggesting that they may be more readily comprehensible and analyzable in comparison to algorithms exhibiting higher cyclomatic complexity.

4.2.2. Evaluation and visualization results

The evaluation and visualization process is a critical stage in the analysis of the results obtained from the measurement of quantum algorithms.

Before starting the automated evaluation, an evaluator thoroughly reviews the measurement results to detect the quality of the evaluation input data. The main objective is to verify that no bugs, significant errors, or anomalies have been detected that could affect the integrity of the data. This preliminary review is essential to ensure the reliability of the results that will be submitted to the automatic evaluation process, as false positives, for example, could be detected after the measurement.

Once the preliminary review has been completed, the automatic evaluation of the product is launched using a Python program. The evaluation is carried out based on the properties and quality model developed based on ISO/IEC 25000 [39], for which (i) the measurement results are processed, and (ii) predefined criteria are applied to

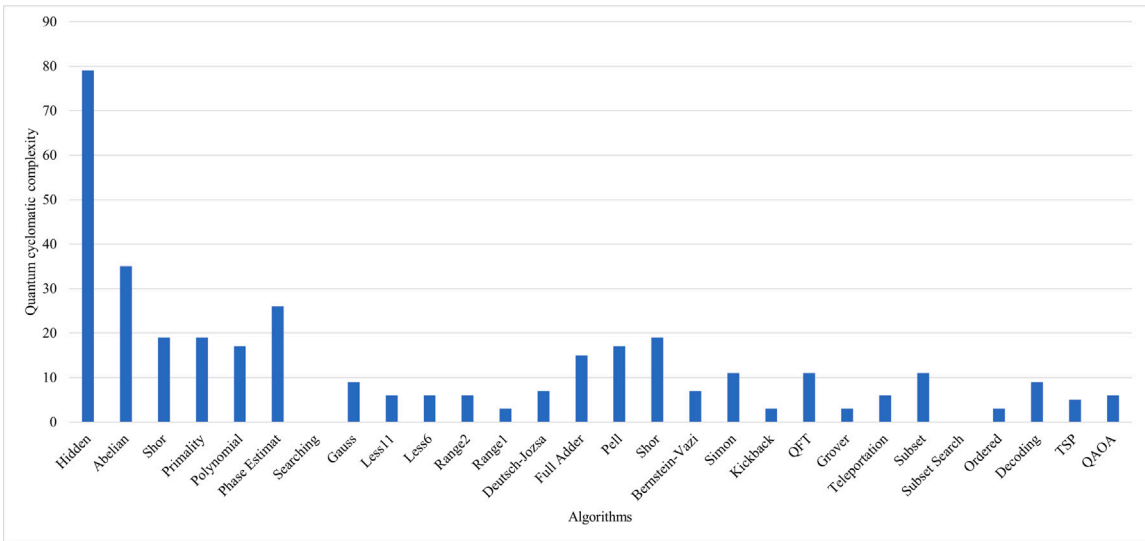


Fig. 5. Quantum cyclomatic complexity measurement results of algorithms developed in Qiskit.

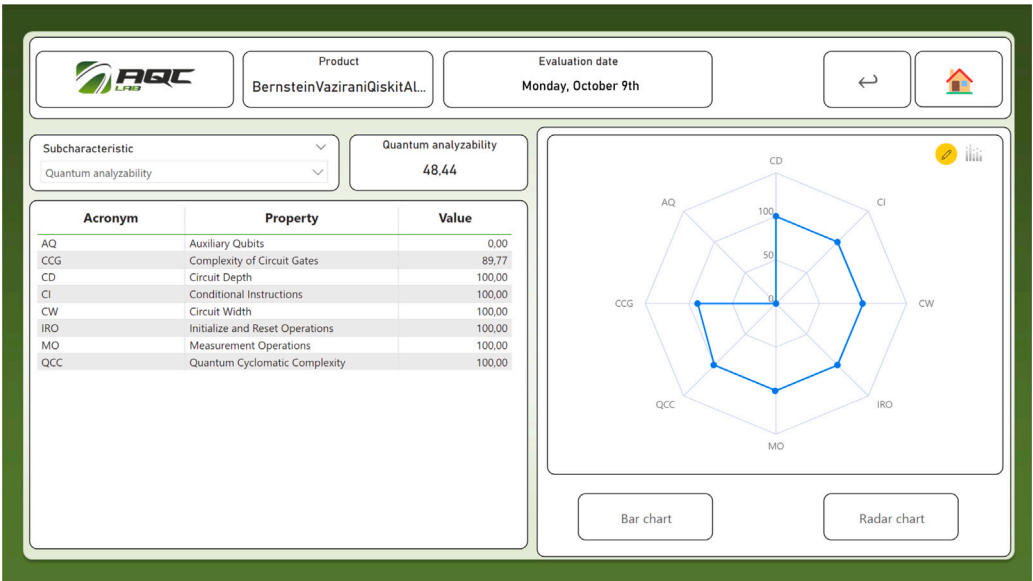


Fig. 6. Evaluation results of the Bernstein-Vazirani algorithm’s analyzability using Power BI.

determine the analyzability of quantum algorithms. During this process, key metrics are calculated and detailed reports are generated that provide a comprehensive view of the analyzability of each algorithm. The results of the final evaluation are captured by a tool developed in Power BI,¹² which enables clear and effective visualization of the data. Power BI offers advanced analysis and visualization capabilities that facilitate the interpretation of the results and enable the identification of relevant trends and patterns. Various interactive dashboards and graphs are used to provide a deep understanding of the analyzability of quantum algorithms. Comparisons can be made between different algorithms, areas for improvement can be identified, and informed decisions can be made about algorithm selection and optimization for specific applications. The results obtained after the evaluation of one of the algorithms using the visualization tool in Power BI are shown below, more specifically the results corresponding to the “Bernstein-Vazirani” algorithm

(see Fig. 6). The results obtained after the evaluation of one of the algorithms using the visualization tool in Power BI are shown below. In particular, the results pertaining to the “Bernstein-Vazirani” algorithm (see Fig. 6) are discussed. Table 1 shows the results obtained for all the properties after the evaluation of each of the 40 algorithm implementations. When analyzing the results, it is observed that some algorithms obtain scores close to the maximum in most of the properties, such as “Hidden Qiskit”, “Abelian Qiskit”, and “Bernstein-Vazirani Qiskit”, among others. This suggests that these algorithms are well-optimized and highly analyzable according to the metrics that evaluate the analyzability properties. On the other hand, some algorithms show areas for improvement, such as “Pell Qiskit”, “Phase Estimation Qiskit”, “Polynomial Qiskit”, “Shor Qiskit” and “Primality Qiskit”. These results indicate that there is room to optimize the resource usage and efficiency of these algorithms. It is interesting to note how the algorithms implemented in Qiskit and Quirk compare to each other. First, the algorithms in Quirk appear to obtain perfect scores on most properties, indicating a high

¹² <https://powerbi.microsoft.com>

Table 1

Evaluation results of key properties for quantum algorithm implementations in Qiskit and Quirk.

Algorithm	CW	CD	CCG	CI	QCC	MO	IRO	AQ
Hidden Qiskit	66	100	24,26	100	100	100	100	100
Abelian Qiskit	100	66	40,38	100	100	100	100	100
Bernstein Qiskit	100	100	89,77	100	100	100	100	0
Bernstein Quirk	100	100	100	100	100	100	100	100
Decoding Qiskit	100	100	12,5	100	100	100	100	100
Deutsch-Jozsa Qiskit	100	100	100	100	100	100	100	100
Deutsch-Jozsa Quirk	100	100	100	100	100	100	100	100
Grover Qiskit	100	100	100	100	100	100	100	100
Grover Quirk	100	100	100	100	100	100	100	100
Full Adder Qiskit	100	100	44,44	100	57,75	100	100	100
Full Adder Quirk	100	100	100	100	100	100	100	100
Gauss Qiskit	100	100	100	100	100	100	100	100
Kickback Qiskit	100	100	98,61	100	100	100	100	100
Kickback Quirk	100	100	100	100	100	100	100	100
Less6 Qiskit	100	100	100	100	100	100	100	100
Less11 Qiskit	100	100	100	100	100	100	100	100
Ordered Qiskit	100	100	100	100	100	100	100	100
Pell Qiskit	100	100	86,86	100	57,75	100	100	100
Pell Quirk	100	100	100	100	100	100	100	100
PE Qiskit	100	66	36,11	100	100	100	100	100
PE Quirk	100	100	100	100	100	100	100	100
Polynomial Qiskit	100	66	79,17	100	57,75	100	100	100
QAOA Qiskit	100	100	91,67	100	100	100	100	100
QAOA Quirk	100	100	100	100	100	100	100	100
QFT Qiskit	100	100	79,17	100	100	100	100	100
QFT Quirk	100	100	100	100	100	100	100	100
Range1 Qiskit	100	100	98,61	100	100	100	100	100
Range2 Qiskit	100	100	100	100	100	100	100	100
Searching Qiskit	100	100	100	100	100	100	100	100
Shor Qiskit	100	66	32,14	100	57,75	100	22	100
Shor Quirk	100	100	100	100	100	100	100	100
Simon Qiskit	100	100	55,68	100	100	100	100	100
Simon Quirk	100	100	100	100	100	100	100	100
Subset Qiskit	100	100	20	100	100	100	100	100
Subset Search Qiskit	100	100	100	66	100	100	100	100
Teleportation Qiskit	100	100	95,83	100	100	100	100	100
Teleportation Quirk	100	100	100	100	100	100	100	100
TSP Qiskit	100	100	84,72	100	100	100	100	100
TSP Quirk	100	100	100	100	100	100	100	100
Primality Qiskit	100	66	32,14	100	57,75	100	22	100

analyzability according to the values obtained by applying the designed evaluation model.

Next, it can also be observed that many of the algorithms achieve perfect scores on the Quantum Cyclomatic Complexity (QCC) property even though some algorithms obtained high values in the measurement (see Fig. 5, Hidden algorithm). This may suggest that the algorithms are well structured and do not have excessive cyclomatic complexity, which is a positive sign in terms of code readability and maintainability. However, the latter may also indicate the need to carry out a review on the model and the tool, to make adjustments on the permissibility thresholds and thus improve the accuracy of the evaluation.

Given the nature of the tool and the context in which these evaluations are being used, there appears to be a trend towards assigning values close to 100 for many properties. This may be due to the way the model and tool are designed to evaluate systems with multiple quantum algorithms at once. When evaluating only one algorithm at a time, it is possible that an adjustment could be made to the model to reflect this situation. It may be useful to consider recalibrating or reconfiguring the model for this specific scenario, which could include adjustments to the classification thresholds. In addition, it is also valuable to consider developing a specific approach to individual algorithm evaluation,

which could involve implementing more detailed evaluation metrics that are sensitive to differences in analyzability.

4.3. Threats to validity

In this study, we propose a process for the generation, deployment and quality assurance of quantum services, combining OpenAPI specification, automation using GitHub Actions, and code quality assessment with SonarQube. Despite the promising results of this study, certain limitations and threats to validity must be acknowledged that could influence the applicability and generalizability of the findings presented.

One of the main risks is that the process was applied primarily to algorithms developed in IBM's Qiskit and Quirk environments. As such, the findings may not be fully generalizable to quantum algorithms developed with other platforms or frameworks, which could limit the overall applicability of the results.

The study evaluated 40 implementations of 28 different quantum algorithms. Nevertheless, it is possible that this set does not fully represent the extensive range of quantum algorithms that exist. The results may vary when applying the evaluation framework to other quantum algorithms, especially those with different structures or complexities or hybrid algorithms.

Also, the SonarQube plugin used in this study is still in its early stages of development. As a result, there may be inaccuracies in measuring certain properties of quantum algorithms, such as cyclomatic complexity or gate complexity.

In this context, although the study proposes specific metrics for analyzing quantum software quality, the metrics chosen may not cover all aspects of quantum software analyzability. It is possible that other relevant metrics or factors have not been included, which could provide further insight into the quality of quantum software.

5. Conclusion

The current level of development in different areas and domains of quantum computing presents a quality problem that needs to be addressed. The lack of mature tools, the specialized knowledge required for the creation of quantum applications, and the differences between quantum providers are some of the aspects that affect quantum quality.

This paper proposes a quality-oriented process for the generation and deployment of quantum services, including the use of static code analysis, to address these challenges. The integration of SonarQube, a widely recognized tool for static code analysis and software quality evaluation, makes it possible to evaluate the quality of the code and to identify potential problems and vulnerabilities. The proposed method also includes using OpenAPI for the definition and generation of quantum services and GitHub Actions for the Continuous Deployment process.

Therefore, the proposed process provides a systematic approach to ensure the quality of quantum software by addressing the challenges identified in the analysis. So, this process for the supervision of the quality of quantum services serves as a foundational step towards establishing standardized practices for assessing and ensuring the quality of quantum software. This is critical as quantum computing becomes more widely adopted in industry and research.

The evaluation and validation of the proposed method shows its effectiveness in generating and deploying quantum services with a high level of quality. After a detailed and individualized evaluation of the set of 40 quantum algorithm implementations using the developed environment, a significant variability in the analyzability of the algorithms is highlighted: while some demonstrate a higher level of overall analyzability in all properties, others reveal some specific areas of improvement as is the case with the quantum cyclomatic complexity property or the complexity of quantum gates.

Considering the individual evaluation of the algorithms, it is recommended to continue working in the presented environment, making adaptations to the model and the tool, possibly through adjustments in the classification thresholds and the introduction of more specific metrics. Furthermore, it is proposed in the future to evaluate combinations of quantum algorithms or even mutations of quantum algorithms, with the aim of better understanding how these variations affect the quality and analyzability of quantum software. These findings highlight the need for further refinement of quantum quality metrics and tools, promoting future research on more accurate and comprehensive methods for the assessment of quantum software.

In conclusion, it is imperative to integrate the generation and deployment process with quality in order to guarantee the reliability and efficiency of quantum software. The proposed quality-oriented approach provides a comprehensive solution to the challenges faced by developers in creating quantum applications. By leveraging the SonarQube parser, OpenAPI, and GitHub Actions, as main tools, the analysis of quantum metrics is integrated into an established development and analysis environment. This integration offers several key advantages, including integration into the existing workflow, ease of use and configuration to adapt to project needs, and easy integration of quantum software quality measurement from any other initiative. These features contribute to the proposal's openness and accessibility.

CRedit authorship contribution statement

Ana Díaz: Investigation, Software, Writing – original draft. **Jaime Alvarado-Valiente:** Investigation, Software, Writing – original draft. **Javier Romero-Álvarez:** Investigation, Software, Writing – original draft. **Enrique Moguel:** Investigation, Methodology, Project administration, Resources, Writing – review & editing. **Jose Garcia-Alonso:** Conceptualization, Funding acquisition, Methodology, Project administration, Writing – review & editing. **Moisés Rodríguez:** Supervision, Validation, Writing – original draft, Writing – review & editing. **Ignacio García-Rodríguez:** Funding acquisition, Methodology, Validation, Writing – review & editing. **Juan M. Murillo:** Conceptualization, Funding acquisition, Project administration, Writing – review & editing.

Funding

This work has been partially funded by MCIN/AEI/10.13039/501100011033 and by the European Union “Next GenerationEU/PRTR”, by the Ministry of Science, Innovation and Universities, Spain (projects PID2021-1240454OB-C31, TED2021-130913B-I00, PDC2022-133465-I00, and PID2021-124054OB-C32). It is also supported by QSERV: Quantum Service Engineering: Development Quality, Testing and Security of Quantum Microservices project funded by the Spanish Ministry of Science and Innovation and ERDF; by European Union under the Agreement — 101083667 of the Project “TECH4E-Tech4efficiencyEDIH” regarding the Call: DIGITAL-2021-EDIH-01 supported by the European Commission through the Digital Europe Program; by the Regional Ministry of Economy, Science and Digital Agenda of the Regional Government of Extremadura (GR21133). And by grant PRE2022-102070, funded by MCIN/AEI/10.13039/501100011033 and by FSE+.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that support the findings of this study are openly available at <https://github.com/javierrome236/quantumDeployment>.

References

- [1] M.J. Klein, Max Planck and the beginnings of the quantum theory, *Arch. Hist. Exact Sci.* 1 (5) (1961) 459–479.
- [2] R. Feynman, Simulating physics with computers, *Internat. J. Theoret. Phys.* 21 (1982) 467–488, <http://dx.doi.org/10.1007/BF02650179>.
- [3] J. Zhao, Quantum software engineering: Landscapes and Horizons, 2020, [arXiv: 2007.07047](https://arxiv.org/abs/2007.07047).
- [4] S. Aaronson, BQP and the polynomial hierarchy, *Proc. Ann. ACM Symp. Theory Comput.* (2009) 141–150, <http://dx.doi.org/10.1145/1806689.1806711>.
- [5] V. Hassija, V. Chamola, V. Saxena, V. Chanana, P. Parashari, S. Mumtaz, M. Guizani, Present landscape of quantum computing, *IET Quantum Commun.* 1 (2) (2020) 42–48, <http://dx.doi.org/10.1049/iet-qtc.2020.0027>.
- [6] A. Ahmad, M. Waseem, P. Liang, M. Fehmideh, A.A. Khan, D.G. Reichelt, T. Mikkonen, Engineering software systems for quantum computing as a service: A mapping study, *Arxiv.* (2023) <http://dx.doi.org/10.48550/arXiv.2303.14713>.
- [7] J. Rojo, D. Valencia, J. Berrocal, E. Moguel, J.M. García-Alonso, J.M. Murillo, Trials and tribulations of developing hybrid quantum-classical microservices systems, *Arxiv.* (2021) <http://dx.doi.org/10.48550/arXiv.2105.04421>.
- [8] E. Moguel, J. Rojo, D. Valencia, J. Berrocal, J. Garcia-Alonso, J.M. Murillo, Quantum service-oriented computing: current landscape and challenges, *Softw. Qual. J.* 30 (4) (2022) 983–1002, <http://dx.doi.org/10.1007/s11219-022-09589-y>.
- [9] A.A. Khan, A. Ahmad, M. Waseem, P. Liang, M. Fahmideh, T. Mikkonen, P. Abrahamson, Software architecture for quantum computing systems—A systematic review, *J. Syst. Softw.* 201 (2023) <http://dx.doi.org/10.1016/j.jss.2023.111682>.
- [10] B. Weder, J. Barzen, F. Leymann, D. Vietz, Quantum software development lifecycle, 2021, [arXiv:2106.09323](https://arxiv.org/abs/2106.09323).
- [11] I. Kumara, W.-J.V.D. Heuvel, D.A. Tamburri, QSOC: Quantum service-oriented computing, 2021, [arXiv:2105.01374](https://arxiv.org/abs/2105.01374).
- [12] J. Alvarado-Valiente, J. Romero-Álvarez, E. Moguel, J. García-Alonso, J.M. Murillo, Technological diversity of quantum computing providers: a comparative study and a proposal for API Gateway integration, *Softw. Qual. J.* (2023) 1–21, <http://dx.doi.org/10.1007/s11219-023-09633-5>.
- [13] S.S. Gill, A. Kumar, H. Singh, M. Singh, K. Kaur, M. Usman, R. Buyya, Quantum computing: A taxonomy, systematic review and future directions, *Softw. - Pract. Exp.* 52 (1) (2022) 66–114, <http://dx.doi.org/10.1002/spe.3039>.
- [14] D. Arias, I.G.R. de Guzmán, M. Rodríguez, E.B. Terres, B. Sanz, J.G. de la Puerta, I. Pastor, A. Zubillaga, P.G. Bringas, Let's do it right the first time: Survey on security concerns in the way to quantum software engineering, *Neurocomputing* 538 (2023) <http://dx.doi.org/10.1016/j.neucom.2023.03.060>.
- [15] A. Mishra, Z. Otaiwi, DevOps and software quality: A systematic mapping, *Comp. Sci. Rev.* 38 (2020) <http://dx.doi.org/10.1016/j.cosrev.2020.100308>.
- [16] M. Piattini, M. Serrano, R. Perez-Castillo, G. Petersen, J.L. Hevia, Toward a Quantum Software Engineering, *IT Prof.* 23 (1) (2021) 62–66, <http://dx.doi.org/10.1109/MITP.2020.3019522>.
- [17] M.A. Akbar, A.A. Khan, S. Mahmood, S. Rafi, Quantum software engineering: A new genre of computing, *Arxiv.* (2022) <http://dx.doi.org/10.48550/arXiv.2211.13990>.
- [18] B. Weder, J. Barzen, F. Leymann, D. Vietz, Quantum software development lifecycle, in: *Quantum Software Engineering*, Springer, 2022, pp. 61–83, http://dx.doi.org/10.1007/978-3-031-05324-5_4.
- [19] I. Kumara, W.-J. Van Den Heuvel, D.A. Tamburri, QSOC: Quantum service-oriented computing, 2021.
- [20] L. Wang, G. Laszewski, M. Kunze, J. Tao, Cloud computing: A perspective study new generation computing, *Adv. Distrib. Inform. Proc.* 28 (2) (2008) 37–146, <http://dx.doi.org/10.1007/s00354-008-0081-5>.
- [21] H. Singh, A. Sachdev, The quantum way of cloud computing, in: 2014 International Conference on Reliability Optimization and Information Technology, ICROIT, IEEE, 2014, pp. 397–400, <http://dx.doi.org/10.1109/ICROIT.2014.6798362>.
- [22] J. Garcia-Alonso, J. Rojo, D. Valencia, E. Moguel, J. Berrocal, J.M. Murillo, Quantum software as a service through a quantum API gateway, *IEEE Internet Comput.* 26 (1) (2021) 34–41, <http://dx.doi.org/10.1109/MIC.2021.3132688>.
- [23] ISO, ISO/IEC 25000, Syst. Softw. Eng. – Systems and software Quality Requirements and Evaluation (SQuaRE) – Guide to SQuaRE (2014).
- [24] ISO, ISO/IEC 25010, Syst. Softw. Eng. – Systems and software Quality Requirements and Evaluation (SQuaRE) – Guide to SQuaRE (2011).
- [25] M. Rodríguez, M. Piattini, C.M. Fernández, A hard look at software quality: Pilot program uses ISO/IEC 25000 family to evaluate, improve and certify software products, *Qual. Prog.* 48 (9) (2015) 30–36.
- [26] A. Díaz, M. Rodríguez, M. Piattini, Towards a set of metrics for hybrid (quantum/classical) systems maintainability, *J.UCS.* Submitted pending publication (2023) <http://dx.doi.org/10.3897/jucs.99348>.
- [27] A. Kumar, Formalization of structural test cases coverage criteria for quantum software testing, *Internat. J. Theoret. Phys.* 62 (2023) <http://dx.doi.org/10.1007/s10773-022-05271-y>.
- [28] M. Rodríguez, J.R. Oviedo, M.G. Piattini, Evaluation of software product functional suitability: A case study, *Softw. Quality Professional Magaz* 18 (2016) URL <https://api.semanticscholar.org/CorpusID:114609549>.

- [29] M. Azeem Akbar, A. Khan, S. Rafi, A systematic decision-making framework for tackling quantum software engineering challenges, *Autom. Softw. Eng.* 30 (2023) <http://dx.doi.org/10.1007/s10515-023-00389-7>.
- [30] M. Piattini, G. Peterssen, R. Pérez-Castillo, J.L. Hevia, M.A. Serrano, G. Hernández, I.G.R. de Guzmán, C.A. Paradelo, M. Polo, E. Murina, et al., The Talavera Manifesto for quantum software engineering and programming, *QANSWER 2020. QuAntum SoftW. Eng & pRogramming*, CEUR-WS: Talavera de la Reina (2020) 1–5.
- [31] P.B. N. Moll, S. Benjamin, Quantum development beyond Qiskit, *npj Quantum Inform* 7 (2021) 1–7.
- [32] A.A. Khan, M.A. Akbar, A. Ahmad, M. Fahmideh, M. Shameem, V. Lahtinen, M. Waseem, T. Mikkonen, Agile practices for quantum software development: Practitioners' perspectives, in: 2023 IEEE International Conference on Quantum Software (QSW), IEEE, 2023, <http://dx.doi.org/10.1109/qsw59989.2023.00012>.
- [33] I.-D. Gheorghe-Pop, N. Tcholtchev, T. Ritter, M. Hauswirth, Quantum DevOps: Towards reliable and applicable NISQ quantum computing, in: 2020 IEEE Globecom Workshops (GC Wkshps), 2020, pp. 1–6, <http://dx.doi.org/10.1109/GCWkshps50303.2020.9367411>.
- [34] S. Schwichtenberg, C. Gerth, G. Engels, From OpenAPI to semantic specifications and code adapters, in: 2017 IEEE International Conference on Web Services, ICWS, IEEE, 2017, pp. 484–491, <http://dx.doi.org/10.1109/ICWS.2017.56>.
- [35] V. Surwase, REST API modeling languages-a developer's perspective, *Int. J. Sci. Technol. Eng* 2 (10) (2016) 634–637.
- [36] J. Romero-Álvarez, J. Alvarado-Valiente, E. Moguel, J. García-Alonso, J.M. Murillo, Using OpenAPI for the development of hybrid classical-quantum services, in: Service-Oriented Computing–ICSOC, 2022, pp. 364–368, http://dx.doi.org/10.1007/978-3-031-26507-5_34.
- [37] W. Khlif, N. Aloui, N.E.B. Ayed, From business process textual description to BPEL models, in: International Conference on Evaluation of Novel Approaches To Software Engineering, Springer, 2021, pp. 112–131, http://dx.doi.org/10.1007/978-3-030-96648-5_6.
- [38] P. Zhao, X. Wu, Z. Li, J. Zhao, Qchecker: Detecting bugs in quantum programs via static analysis, 2023, [arXiv:2304.04387](https://arxiv.org/abs/2304.04387), URL <https://arxiv.org/abs/2304.04387>.
- [39] M. Rodríguez, M. Piattini, Software product quality evaluation using ISO/IEC 25000, *ERCIM News* 2014 (99) (2014) URL <http://ercim-news.ercim.eu/en99/special/software-product-quality-evaluation-using-iso-iec-25000>.